

Objects and Classes

Mao

Outline



- **Object-oriented Programming**
- **Defining Classes for Objects**
- **Constructors**
- **Constructing and Using Objects**
- **Separating Class Definition from Implementation**
- **Preventing Multiple Inclusions**
- **Inline Functions in Classes**
- **Data Field Encapsulation**

Outline



- **The `string` Class**
- **Array of Objects**
- **Instance and Static Members**
- **Constant Member Functions**

Object-oriented Programming



When developing large-scale software systems, using selections, loops, functions, and arrays may not be sufficient enough.

Object-oriented programming (OOP) is a programming paradigm that organizes code around data, or **objects**, **rather than** functions and logic.

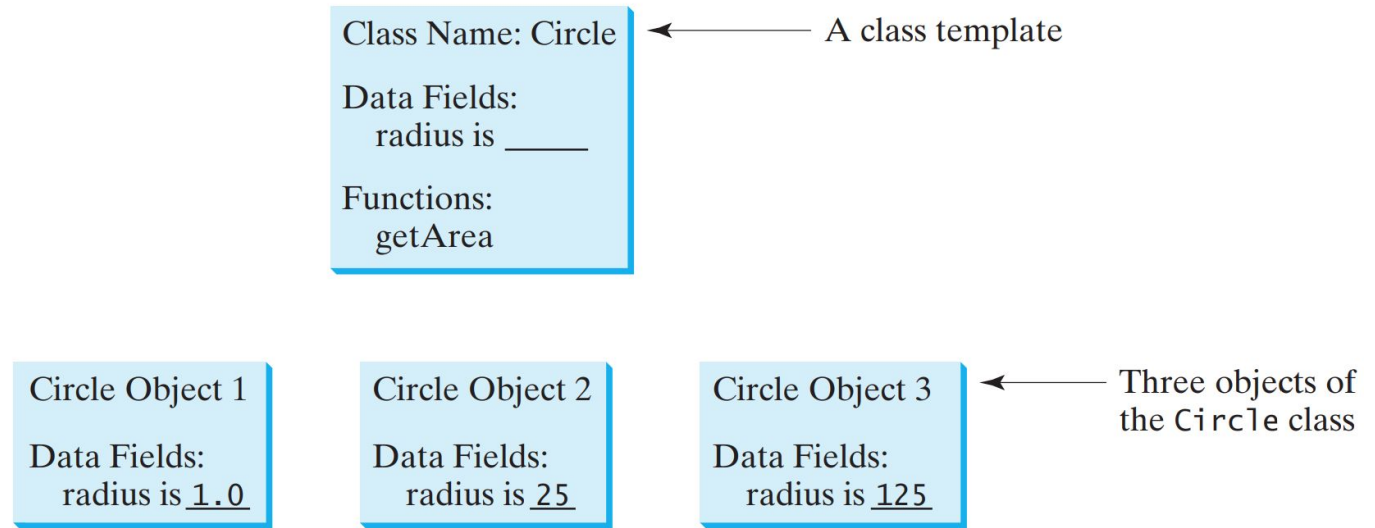
An object represents an entity in the real world that can be distinctly identified. For example, **a student, a desk, a circle, a button**, and even a loan can all be viewed as objects.

Defining Classes for Objects

An object has a unique identity, state, and behavior.

- The state of an object (also known as **properties or attributes**) is represented by **data fields with their current values**. A circle object, for example, has a data field, **radius**, which is the property that **characterizes a circle**.
- The behavior of an object (also known as **actions**) is **defined by functions**. To invoke a function on an object is to ask the object to perform an action. For example, a function named `getArea()` may be defined for circle objects. A circle object may invoke `getArea()` to return its area.

Defining Classes for Objects



A class is a blueprint for creating objects.

Defining Classes for Objects



The relationship between classes and objects:

- Objects of **the same type** are defined using **a common class**.
- **A class is a template, blueprint, or contract that defines what an object's data fields and functions will be.**
- An object is **an instance** of a class.
- Many instances can be created using a class. Creating an instance is referred to as instantiation.
- The terms object and instance are often interchangeable.

Defining Classes for Objects

A C++ class uses **variables to define data fields** and **functions to define behaviors**.

Additionally, a class provides **functions of a special type**, known as **constructors**, which are **invoked when a new object is created**. A constructor is a special kind of function.

Constructors can **perform any action**, but they are **designed to initialize actions**, such as initializing the data fields of objects.


```
class Circle
{
public:
    // The radius of this circle
    double radius; ← Data field

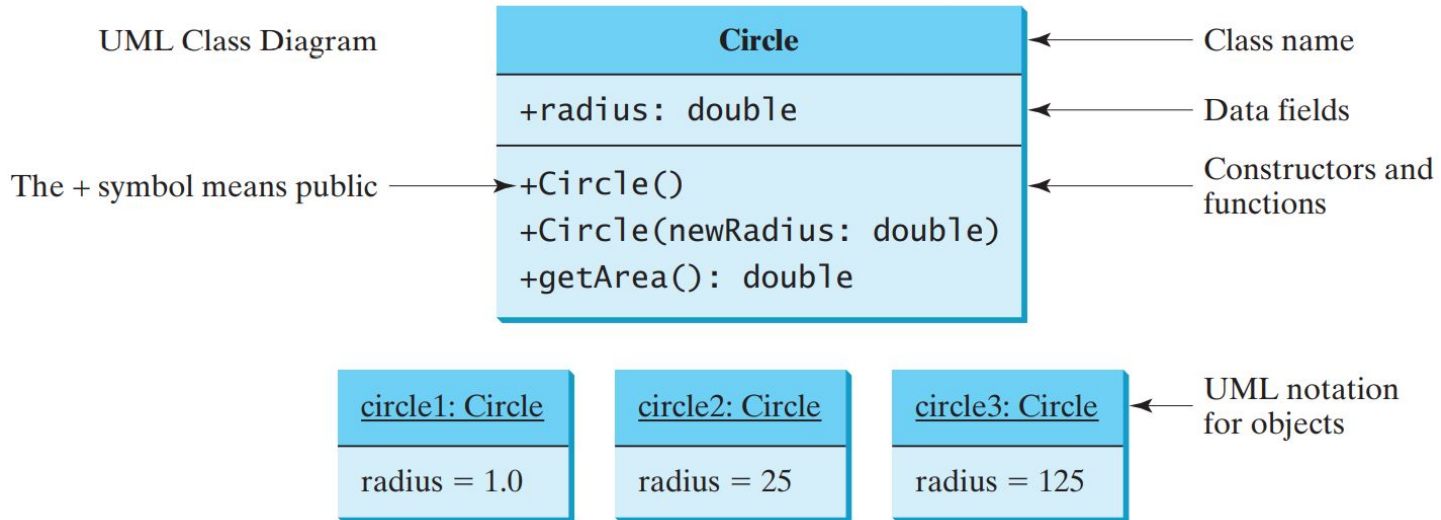
    // Construct a circle object
    Circle()
    {
        radius = 1;
    }

    // Construct a circle object
    Circle(double newRadius) ← Constructors
    {
        radius = newRadius;
    }

    // Return the area of this circle
    double getArea() ← Function
    {
        return radius * radius * 3.14159;
    }
};
```

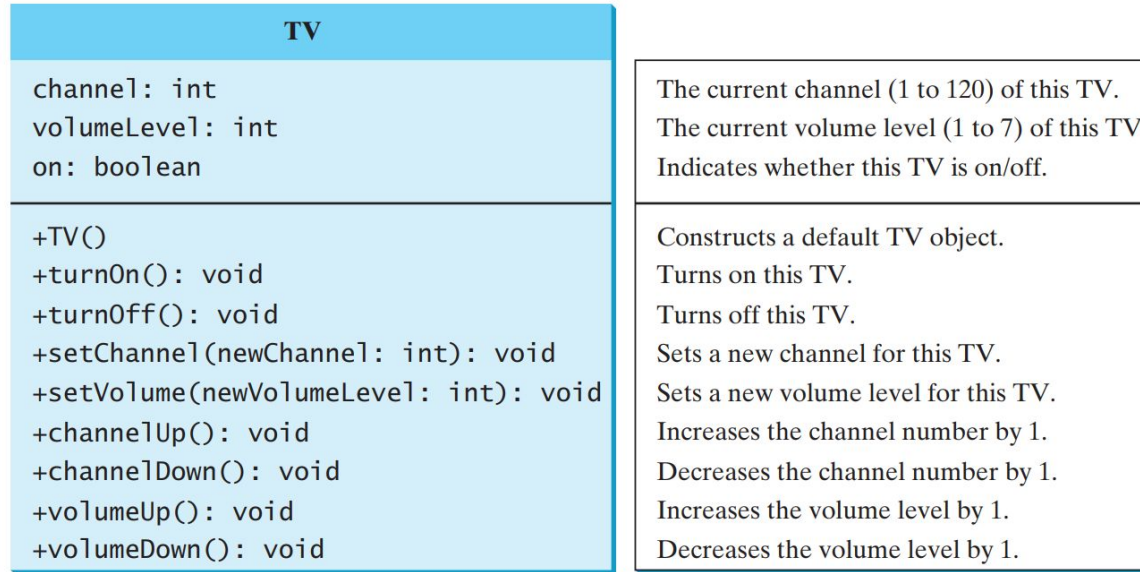
An example of the class for Circle objects

The illustration of class and objects in the previous slide can be standardized using UML (Unified Modeling Language) notation, called a UML class diagram, or simply class diagram.



Classes and objects can be represented using UML notations.

As another example, consider TV sets. Each TV is an object with state (**current channel, current volume level, power on or off**) and behaviors (**change channels, adjust volume, turn on/off**). TV sets can modeled using a class. The UML diagram for the class is shown below.



TV.cpp

The TV class models TV sets.

Constructors



A constructor is invoked to create an object, and it is a special kind of function, with three peculiarities:

- **Must have the same name as the class itself.**
- **No return type—not even void.**
- **Invoked when an object is created.** Constructors play the role of initializing objects.

Constructors

It is a **common mistake** to put the void keyword in front of a constructor.
For example,

```
void Circle()  
  
{  
  
}
```

Most C++ compilers will report an error, but some will treat this as a regular function, not as a constructor.

Constructors are for initializing data fields.

In C++ 11, a member data field can be initialized upon its declaration. This is known as member initializer in C++ 11.

```
class Circle
{
public:
    // The radius of this circle
    double radius = 5.0;
    Circle()
    {
        radius = 1.0; // This one takes precedence.
    }
    ...
}
```

If the same data member has both a class member initializer and a mem-init in the constructor, the latter takes precedence.

Constructing and Using Objects

A constructor is invoked when an object is created. The syntax to create an object using the no-argument constructor is

```
ClassName objectName;
```

```
Circle circle1; // Circle circle1(); is not correct.
```

```
Circle circle1 = Circle();
```

The syntax to create an object using a constructor **with arguments** is

```
ClassName objectName(arguments);
```

```
Circle circle2(5.5);
```

```
Circle circle2 = Circle(5.5)
```

Constructing and Using Objects

After an object is created, its data can be accessed and its functions invoked using the **dot operator (.)**, also known as **the object member access operator**:

`objectName.dataField; //references a data field in the object.`

`circle1.radius`

`objectName.function(arguments); //invokes a function on the object.`

`circle1.getArea()`

Constructing and Using Objects

Data fields may be initialized in the constructor using an initializer list in the following syntax:

```
ClassName(parameterList): datafield1(value1), datafield2(value2) // Initializer list
{
    // Additional statements if needed
}
```

```
Circle::Circle() : radius(1.0)
{
}
```

=

```
Circle::Circle()
{
    radius = 1.0;
}
```

Note that:

- The class names in the C++ library are named in lowercase.
- The objects are named like variables.
- For a custom class, **capitalize the first letter of each word** in a class name—for example, the names Circle, GeometricObject, and Desk.
- The **assignment operator =** can be used to copy the contents from one object to the other. By default, **each data field of one object is copied to its counterpart in the other object**. For example,

```
circle2 = circle1;
```

copies the radius in circle1 to circle2. After the copy, circle1 and circle2 are **still two different objects but have the same radius**.

TestClassAssignmentOperator.cpp

- Object names are like array names. Once an object name is declared, it represents an object. It **cannot be reassigned to represent another object**. In this sense, an object name is **a constant**, though **the contents of the object may change**.
- Memberwise copy can change an object's contents but not its name.
- **Data are physically stored in an object**, but functions are not. Since **functions are shared** by all objects of the same class, the compiler creates **just one copy for sharing**. The actual size of an object can found using the **sizeof** function:

```
Circle circle1;  
cout << sizeof(circle1) << endl;
```

Separating Class Definition from Implementation



C++ allows to separate class definition from implementation.

- **The class definition describes the contract (all the data fields, constructor prototypes, and function prototypes) of the class .**
- **The class implementation carries out the contract.**

Circle.h

Circle.cpp

TestCircleWithHeader.cpp

Separating Class Definition from Implementation

To compile a main program from the command line, just add all its supporting files in the command. For example, to compile `TestCircleWithDefinition.cpp` using a GNU C++ compiler, the command is

`g++ Circle.cpp TestCircleWithHeader.cpp -o a.out`

Replace the function name

`int TestCircleWithHeader()`

in `TestCircleWithHeader.cpp` with

`int main()`

so that there will be a main function to start the program.

Preventing Multiple Inclusions

It is **a common mistake** to include, inadvertently, **the same header file in a program multiple times**. Suppose Head.h includes Circle.h and TestHead.cpp includes both Head.h and Circle.h, as shown below:

Head.h

```
#include "Circle.h"  
// Other code in Head.h omitted
```

TestHead.cpp

```
#include "Circle.h"  
#include "Head.h"  
  
int main()  
{  
    // Other code in TestHead.cpp omitted  
}
```

Preventing Multiple Inclusions



When compiling `TestHead.cpp`, there will be a **compile error indicating that there are multiple definitions for `Circle`**.

C++ preprocessor inserts the contents of the header file at the position where the header is included.

In the previous slide, the preprocessor will **add the definition for the `Circle` class twice as result of including both `Circle.h` and `Head.h` in `TestHead.cpp`**, which causes the multiple-inclusion errors.

Preventing Multiple Inclusions

To avoid multiple-inclusion errors, define a class using the following template and convention for naming the symbol:

```
#ifndef ClassName_H  
#define ClassName_H
```

A class header for the class named `ClassName`

```
#endif
```



```
class A
{
public:
    A()
    {
        // Do something;
    }
    double f1()
    {
        // Return a number
    }
    double f2();
};

// Implement function as inline
inline double A::f2()
{
    // Return a number
}
```

When a function is implemented **inside a class definition**, it automatically becomes an inline function. This is also known as **inline definition**.

For example, in the following definition for class A, the constructor and function **f1** are **automatically inline functions**, but function f2 is not.

Data Field Encapsulation

To prevent direct modifications of properties, declare the data field private using the **private** keyword. This is known as data field encapsulation.

Properties are private by default.

To make the radius data field private in the **Circle** class, the class can be defined as shown on the right.

```
class Circle
{
    public:
        Circle();
        Circle(double);
        double getArea();
    private:
        double radius;
};
```

Data Field Encapsulation

A private data field **cannot be accessed by an object through a direct reference outside the class** that defines the private field.

- To make a private data field accessible, provide a **get function** to return the field's value.

returnType getPropertyNames();

- If the returnType is bool, by convention the get function should be defined as follows:

bool isPropertyName();

CircleWithPrivateDataFields.h

CircleWithPrivateDataFields.cpp

TestCircleWithPrivateDataFields.cpp

- To enable a private data field to be updated, provide a **set function to set a new value**.

void setPropertyName(dataType propertyValue)

Data fields and functions can be in any order in a class. For example, all the following declarations are the same:

```
class Circle
{
public:
    Circle();
    Circle(double);
    double getArea();
    double getRadius();
    void setRadius(double);

private:
    double radius;
};
```

(a)

```
class Circle
{
public:
    Circle();
    Circle(double);

private:
    double radius;

public:
    double getArea();
    double getRadius();
    void setRadius(double);
};
```

(b)

```
class Circle
{
private:
    double radius;

public:
    double getArea();
    double getRadius();
    void setRadius(double);

public:
    Circle();
    Circle(double);
};
```

(c)

Example: Loan class

$$\text{monthly payment} = \frac{r}{1 - (1 + r)^{-N}} P$$

- **r** - the monthly interest rate, simply the yearly percentage rate divided by 12,
- **N** - the number of monthly payments, called the loan's term, and
- **P** - the amount borrowed, known as the loan's principal.

Example: Loan class

Loan.h Loan.cpp TestLoanClass.cpp

Loan	
-annualInterestRate: double -numberOfYears: int -loanAmount: double	The annual interest rate of the loan (default: 2.5). The number of years for the loan (default: 1) The loan amount (default: 1000).
+Loan() +Loan(rate: double, years: int, amount: double) +getAnnualInterestRate(): double +getNumberOfYears(): int +getLoanAmount(): double +setAnnualInterestRate(rate: double): void +setNumberOfYears(years: int): void +setLoanAmount(amount: double): void +getMonthlyPayment(): double +getTotalPayment(): double	Constructs a default loan object. Constructs a loan with specified interest rate, years, and loan amount. Returns the annual interest rate of this loan. Returns the number of the years of this loan. Returns the amount of this loan. Sets a new annual interest rate to this loan. Sets a new number of years to this loan. Sets a new amount to this loan. Returns the monthly payment of this loan. Returns the total payment of this loan.

The Loan class models the properties and behaviors of loans.

The string Class

In C++ there are two ways to process strings.

- One way is to treat them as arrays of characters ending with the null terminator ('\0'), as discussed “C-Strings.” These are known as C-strings. The null terminator indicates the end of the string, which is important for the C-string functions to work.

```
string s = "Welcome to C++";
```

- The other way is to process strings using the string class.

```
string s("Welcome to C++");
```

The string Class

To create an empty string using `string`'s no-arg constructor. For example,

```
string s;
```

To create a string from a C-string using `string`'s constructor is shown in the following code:

```
char s1[] = "Good morning";
```

```
string s(s1);
```

Here `s1` is a C-string and `s` is a string object.

The string Class

string

```
+append(s: string): string  
+append(s: string, index: int, n: int): string  
  
+append(s: string, n: int): string  
+append(n: int, ch: char): string
```

Appends string s into this string object.

Appends n number of characters in s starting at the position index to this string.

Appends the first n number of characters in s to this string.

Appends n copies of character ch to this string.

string

```
+assign(s[]: char): string  
+assign(s: string): string  
+assign(s: string, index: int, n: int): string  
  
+assign(s: string, n: int): string  
+assign(n: int, ch: char): string
```

Assigns array of characters or a string s to this string.

Assigns string s to this string.

Assigns n number of characters in s starting at the position index to this string.

Assigns the first n number of characters in s to this string.

Assigns n copies of character ch to this string.

The string Class

string

```
+at(index: int): char  
+clear(): void  
+erase(index: int, n: int): string  
+empty(): bool
```

Returns the character at the position index from this string.

Removes all characters in this string.

Removes n characters from this string starting at position index.

Returns true if this string is empty.

string

```
+length(): int  
+size(): int  
+capacity(): int  
+c_str(): char[]  
+data(): char[]
```

Returns the number of characters in this string.

Same as length().

Returns the size of the storage allocated for this string.

Returns a C-string for this string.

Same as c_str().

string

```
+compare(s: string): int  
  
+compare(index: int, n: int, s: string): int
```

Returns a value greater than 0, 0, or less than 0 if this string is greater than, equal to, or less than s.

Compares this string with substring s(index, ..., index + n-1).

The string Class

string

`+substr(index: int, n: int): string`

`+substr(index: int): string`

Returns a substring of n characters from this string starting at position index.

Returns a substring of this string starting at position index.

string

`+find(ch: char): unsigned`

`+find(ch: char, index: int): unsigned`

`+find(s: string): unsigned`

`+find(s: string, index: int): unsigned`

Returns the position of the first matching character for ch.

Returns the position of the first matching character for ch at or from the position index.

Returns the position of the first matching substring s.

Returns the position of the first matching substring s starting at or from the position index.

String

`+insert(index: int, s: string): string`

`+insert(index: int, n: int, ch: char): string`

`+replace(index: int, n: int, s: string): string`

Inserts the string s into this string at position index.

Inserts the character ch n times into this string at position index.

Replaces the n characters starting at position index in this string with the string s.

<i>Operator</i>	<i>Description</i>
<code>[]</code>	Accesses characters using the array subscript operator.
<code>=</code>	Copies the contents of one string to the other.
<code>+</code>	Concatenates two strings into a new string.
<code>+=</code>	Appends the contents of one string to the other.
<code><<</code>	Inserts a string to a stream
<code>>></code>	Extracts characters from a stream to a string delimited by a whitespace or the null terminator character.
<code>==, !=, <,</code>	Six relational operators for comparing strings.
<code><=, >, >=</code>	

- **Converting Numbers to Strings**
- **Splitting Strings**
- **Replacing Strings**

TestStringClass.cpp

Array of Objects

TestArrayOfObejctsTotalArea.cpp

The following statement declares an array of 10 Circle objects:

```
Circle circleArray[10]; // Declare an array of ten Circle objects
```

The name of the array is `circleArray`, and the no-arg constructor is called to initialize each element in the array.

The array initializer can also be used to declare and initialize an array using a constructor with arguments. For example,

```
Circle circleArray[3] = {Circle(3), Circle(4), Circle(5)};
```

Instance and Static Members

The data fields used in the classes so far are known as **instance data fields, or instance variables**. An instance variable is **tied to a specific instance** of the class; it is **not shared among objects** of the same class.

To declare a static variable or a static function, put the modifier **static** in the variable or function declaration:

```
static int numberOfObjects;
```

```
static int getNumberOfObjects();
```

Instance and Static Members

Static:

- Static variables and functions can be accessed without creating objects.
- Static data fields and functions can be accessed from the instances of the class, **but** it is better to **access them from the class name—e.g., Circle::**.

CircleWithStaticDataFields.h

CircleWithStaticDataFields.cpp

TestCircleWithStaticDataFields.cpp

Constant Member Functions

The **const** keyword can be used to specify a **constant parameter** to tell the compiler that **the parameter should not be changed** in the function.

The **const** keyword can also be used to specify a **constant member function** (or simply constant function) to tell the compiler that the function **does not change the data fields in the object**.

```
class CircleS
{
    public:
        ...
        double getArea() const;
        double getRadius();
        ...
}
```

Check **CircleS::getArea()** function in **CircleWithPrivateDataFields.cpp**

Reference



- **Revel for Introduction to C++ Programming and Data Structures by Daniel Liang, 5th edition, Pearson, 2022**